AI-Powered Interactive Storytelling Platform

Project Proposal

By Farhad Azadjoutabar
Vimala Nagireddygar
Yiming Zeng

CS 7319 – Group 5

Table of Contents

Project Team Information 1
Main Capabilities of the Software System 1
- Fictional World and Campaign Creation 1
- AI-Driven Interactive Gameplay 1
- Player Character and Game State Management 2
- Virtual Object Creation and Marketplace 2
- Payment Processing and Subscription Management 2
- Player Behavior Tracking and Campaign Analytics 2
- AI-Based Campaign Evaluation and Recommendations 3
- AI Model Integration and Management 3
- Data Storage and Management 3
- User and Platform Administration and Future Extensibility 3
- Summary 4

Expected User Interactions, Behaviors, Outputs, and Results 5
- Campaign Author Interactions 5
- Player Interactions and Gameplay 5
- AI Game Director Behavior 5
- Object Creator and Marketplace Interactions 5
- Campaign Analytics and Reporting 6
- AI-Based Campaign Recommendations 6
- Subscription and AI Model Interactions 6
- System Administration and Background Operations 6
- Overall Interaction Scenario 7

Prototype Interface Screens 8
Architecture 10
- The two chosen architectures 10
- Layer Style Component Interaction and Communication Flow 12
- Event-based Style Component Interaction and Communication Flow 13
- Final Decision 14

Implementation Plan 15

Project Team Information

Final Project Group 5

Student 1	
First Name: Farhad	Last Name: Azadjoutabari
CS 7319	On Campus
Student 2	
First Name: Yiming	Last Name: Zeng
CS 7319	On Campus
Student 3	
First Name: Vimala	Last Name: Nagireddygari
CS 7319	On Campus

Main Capabilities of the Software System

The proposed software is an AI-powered, text-based role-playing platform that allows content creators to design and publish fictional worlds, players to explore and interact with those worlds, and object creators to develop and sell virtual assets. The platform combines AI-driven storytelling, campaign management, a digital marketplace, and behavioral analytics to create personalized gaming experiences and support continuous campaign improvement.

Fictional World and Campaign Creation

The software allows content creators, including writers, game designers, and filmmakers, to develop fictional worlds with unique environments, storylines, characters, relationships, objects, and rules. The system supports two approaches to campaign creation:

- **Document-based creation:** Authors can upload a document describing their fictional world. An AI assistant analyzes the content, identifies its main elements, and evaluates whether it contains sufficient information for publication.
- **Interactive creation:** Authors can build campaigns directly through the user interface, defining characters, environments, objects, and rules while receiving AI-generated suggestions.

In both approaches, the AI assistant identifies missing or inconsistent information and recommends improvements. Authors can revise their campaigns through an iterative process until they meet the minimum publication requirements.

Once a campaign is ready, the author can publish it with specific completion conditions, such as a five-month duration or a minimum of 200 participating players. When either target is reached, the campaign closes to new participation. Authors can then review its performance, make improvements, and publish an updated version as a new campaign.

AI-Driven Interactive Gameplay

The central functionality of the platform is its AI-powered, text-based role-playing environment. Players can browse published campaigns, select a fictional world, and create their characters according to the campaign's rules and instructions. They then interact with the world through text messages describing their actions, decisions, and conversations.

An AI Game Director manages gameplay by interpreting player messages, considering the world's rules and current conditions, evaluating possible actions, determining their outcomes, and generating narrative responses.

Unlike traditional games that rely primarily on predefined outcomes, the AI Director can make dynamic and stochastic decisions based on the context of each interaction. For example, when a player attempts to deceive a fictional character, the AI considers the character's personality, relationships, previous conversations, and campaign instructions before determining the response.

This approach allows different players to experience unique journeys, even when participating in the same fictional world.

Player Character and Game State Management

Each player has an independent game session. Although multiple players can participate in the same campaign, their actions and decisions only affect their individual game states.

The system maintains persistent records of player characters, attributes, inventories, relationships, visited locations, conversations, decisions, and important events. Changes to the environment or characters are also preserved throughout the player's journey.

For example, if a player acquires an object, destroys a building, or changes a relationship with another character, the system records the resulting state changes. The AI Game Director uses this information during future interactions to maintain narrative continuity.

Player histories and game states are stored in a NoSQL database, allowing users to resume their adventures without losing previous progress.

Virtual Object Creation and Marketplace

The platform includes a virtual marketplace where object creators can design and sell assets such as weapons, clothing, buildings, creatures, and fictional characters. Each asset may contain descriptions, properties, and behavioral instructions that determine how it functions within a campaign.

Campaign authors can browse the marketplace and purchase objects to incorporate into their fictional worlds instead of creating every asset from scratch.

The marketplace uses a campaign-specific licensing model, meaning that a purchased object can only be used in one campaign. Authors must purchase a separate license to use the same object in another campaign.

Marketplace participation is optional, and content creators can develop their entire fictional worlds independently.

Payment Processing and Subscription Management

The software follows a subscription-based business model with different service levels for players and content creators.

Players can select from Basic, Pro, and Plus subscription tiers, which provide access to different AI models and service capabilities. Higher tiers may offer more advanced models with stronger reasoning, contextual understanding, and narrative generation abilities.

Content creators have separate subscription options that determine their access to AI assistance, content evaluation, campaign analytics, and recommendations.

The payment subsystem manages recurring subscriptions and marketplace transactions. Object creators can connect their payment accounts to receive payments for purchased assets. Premium subscription plans may also provide marketplace credits for content creators.

Player Behavior Tracking and Campaign Analytics

During gameplay, the system records structured events such as character interactions, location visits, item acquisitions, player decisions, and action outcomes. Each event is associated with the relevant campaign and individual game session. The analytics component aggregates these records across player

sessions to identify behavioral patterns and calculate performance measures, including character encounter rates, exploration rates, and action success rates.

Analytics reports are generated asynchronously at predefined milestones or after a campaign closes. Campaign authors can use these reports to evaluate whether their fictional worlds perform as intended and identify opportunities for improvement. The collected information also supports AI-based recommendations for future campaign versions without modifying the original published campaign or other players' game states.

AI-Based Campaign Evaluation and Recommendations

The platform uses AI to interpret campaign analytics and recommend improvements based on actual player behavior and the author's original design objectives.

For example, if most players easily deceive a character intended to be difficult to manipulate, the AI may recommend modifying that character's personality or behavioral instructions. Similarly, if an important location is rarely discovered, the system may suggest changing its placement, introducing clues, or modifying the environment.

Recommendations may address character relationships, environmental descriptions, object properties, interaction difficulty, and narrative progression.

The author reviews these suggestions and decides which changes to incorporate into future campaign versions. This creates a continuous improvement process based on measurable player experiences.

AI Model Integration and Management

The system supports integration with multiple large language model providers, including OpenAI, Anthropic, Google, and Meta, rather than depending on a single AI service.

A dedicated AI integration component manages communication with external models and selects appropriate services based on the requested functionality and user's subscription level.

Different AI models can support campaign creation, content validation, gameplay management, conversation analysis, and recommendation generation.

This flexibility allows the platform to adopt newer AI models or change providers without requiring major modifications to its core functionality, while balancing performance, operational costs, and subscription benefits.

Data Storage and Management

The software uses a NoSQL database as its primary storage solution for campaign definitions, world structures, characters, objects, relationships, natural-language rules, user profiles, game sessions, player histories, and structured gameplay events.

NoSQL storage provides flexibility because fictional worlds may have significantly different structures and relationships.

The system separates the original campaign definition from each player's individual game state, ensuring that changes within one session do not affect other players.

Stored information supports the AI Game Director during gameplay and provides historical data for campaign analytics and reporting.

User and Platform Administration and Future Extensibility

The platform supports multiple user roles, including players, campaign authors, object creators, and administrators. Each role has specific permissions and access to relevant functionalities.

Administrative capabilities include user management, subscription oversight, content moderation, and marketplace administration. Some of these functions may remain outside the initial implementation scope.

Although the primary gameplay is single-player, the architecture should also consider future multiplayer support, where multiple players could interact within a shared game session.

Other potential extensions include additional AI providers, new marketplace asset categories, and more advanced analytics tools. Therefore, modularity, scalability, maintainability, and extensibility will be important considerations during architectural evaluation.

Summary

Overall, the proposed software combines AI-driven gameplay, fictional world creation, digital asset trading, persistent player state management, and behavioral analytics in a single platform.

Its main distinguishing feature is the integration of an intelligent AI Game Director with a campaign evaluation and improvement process. Players can experience dynamic and personalized stories, while content creators can observe how their fictional worlds perform and use real gameplay data to improve future versions.

The system is designed to support multiple categories of users, different AI models, and complex relationships between its components. These characteristics make it a suitable candidate for evaluating alternative software architectural styles, particularly with respect to modularity, scalability, maintainability, data management, and extensibility.

Expected User Interactions, Behaviors, Outputs, and Results

The proposed software supports four main user roles: players, campaign authors, object creators, and system administrators. Each interacts with different system components according to their role and permissions. The platform uses AI models, a NoSQL database, and supporting services to process requests, manage information, and generate appropriate responses.

Campaign Author Interactions

Campaign authors create and manage fictional worlds by uploading a descriptive document or developing the campaign directly through the user interface. During creation, an AI assistant evaluates characters, environments, rules, objects, and relationships, identifies missing or inconsistent information, and recommends improvements. Authors can revise their campaigns until they satisfy publication requirements.

Once ready, authors publish their campaigns with predefined completion conditions, such as a five-month duration or 200 participating players. When either target is reached, the campaign closes to new participation, allowing authors to review its performance and prepare an improved version.

Expected outputs: Structured campaigns, AI-generated feedback, publication confirmations, campaign status updates, and access to analytical reports.

Player Interactions and Gameplay

Players browse published campaigns, select a fictional world, and create characters according to its rules. Gameplay takes place through a text-based interface where players communicate with fictional characters, explore locations, and describe their actions.

For example, a player may write: *"I approach the castle guard and try to convince him that I am a messenger from the king."*

The AI Game Director interprets the request, considers relevant campaign rules, character information, previous interactions, and current conditions, and determines the outcome. Its decisions can be stochastic rather than entirely predefined, allowing different players to experience unique consequences.

Each player has an independent session, and the system preserves their decisions, character information, and changes to the fictional world. Players can resume their adventures without losing progress.

Expected outputs: AI-generated narrative responses, character conversations, action outcomes, updated game states, and persistent gameplay histories.

AI Game Director Behavior

The AI Game Director coordinates gameplay and maintains narrative continuity. For each player message, it identifies the intended action, retrieves relevant campaign information and session history, evaluates possible outcomes, and generates an appropriate response.

The system then records resulting changes to the player's state. For example, if the AI determines that a player successfully steals an object, the system updates the player's inventory and stores the event for future interactions.

Expected result: A dynamic, context-aware gaming experience that combines unpredictable narrative decisions with consistent and persistent game-state management.

Object Creator and Marketplace Interactions

Object creators develop and publish virtual assets, including weapons, clothing, buildings, creatures, and fictional characters. Each object contains descriptions and properties that determine its potential use within a campaign.

Campaign authors can browse the marketplace and purchase these assets. The payment component processes transactions and grants usage rights for one specific campaign. Authors who wish to reuse an object in another campaign must purchase it again. Alternatively, they can create their own assets without using the marketplace.

Expected outputs: Published asset listings, purchase confirmations, campaign-specific usage rights, transaction records, and creator revenue information.

Campaign Analytics and Reporting

During gameplay, the system records structured events such as character encounters, location visits, item acquisitions, decisions, and action outcomes.

At predefined milestones or campaign completion, the analytics component processes these records and generates performance reports. For example, a report may indicate that 95% of players encountered Character A, 87% of those encounters resulted in successful deception, and only 11% discovered Character B.

These results help authors understand player behavior and identify campaign elements that may not perform as intended. Premium subscriptions may also support AI-based analysis of complete conversations for deeper behavioral insights.

Reports are generated asynchronously without interrupting active gameplay.

Expected outputs: Participation statistics, character interaction rates, exploration patterns, action success rates, and campaign performance reports.

AI-Based Campaign Recommendations

The AI recommendation component analyzes campaign performance and suggests improvements based on observed player behavior.

For example, if players easily deceive a character intended to be difficult to manipulate, the AI may recommend modifying that character's behavior. Similarly, if an important location is rarely discovered, it may suggest relocating it or introducing additional clues.

Campaign authors review these recommendations and decide which modifications to include in future versions.

Expected outputs: Recommendations for improving characters, environments, interactions, difficulty, and overall campaign design.

Subscription and AI Model Interactions

Players and content creators select subscription plans that determine their available features and AI services. The Basic, Pro, and Plus tiers provide different levels of access to language models and platform capabilities.

When a user submits an AI-related request, the system selects an eligible model from supported providers, such as OpenAI, Anthropic, Google, or Meta, based on the subscription and requested functionality.

Content creators may also receive different levels of AI assistance, analytics, and recommendations.

Expected outputs: Subscription confirmations, available AI model options, access to eligible services, usage information, and payment records.

System Administration and Background Operations

Administrators manage user accounts, content moderation, marketplace activities, and subscription-related issues.

Meanwhile, background components handle data storage and retrieval, independent player-session states, payment processing, campaign completion monitoring, and report generation. The NoSQL database supports flexible campaign structures and preserves the information required for gameplay and analytics.

Expected outputs: Updated records, administrative confirmations, campaign status changes, transaction information, and completed background processing results.

Overall Interaction Scenario

Consider a filmmaker who creates a fictional city to evaluate how players respond to its characters and mysteries.

The filmmaker defines the world, optionally purchases marketplace objects, and uses the AI assistant to validate the campaign before publication. Players then enter the world, create characters, and interact with the AI Game Director through text-based conversations.

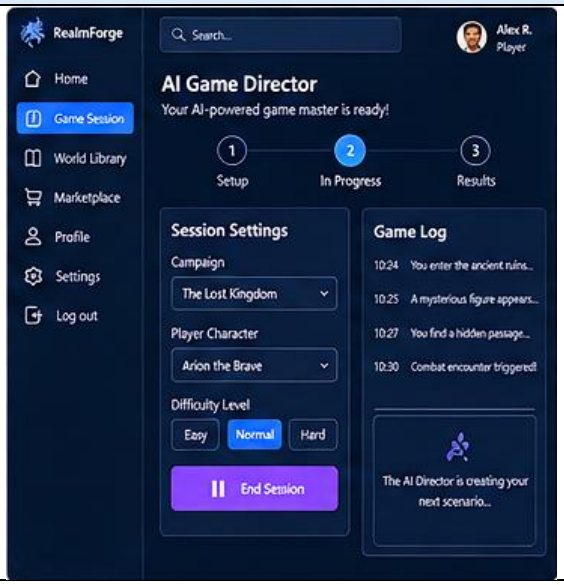
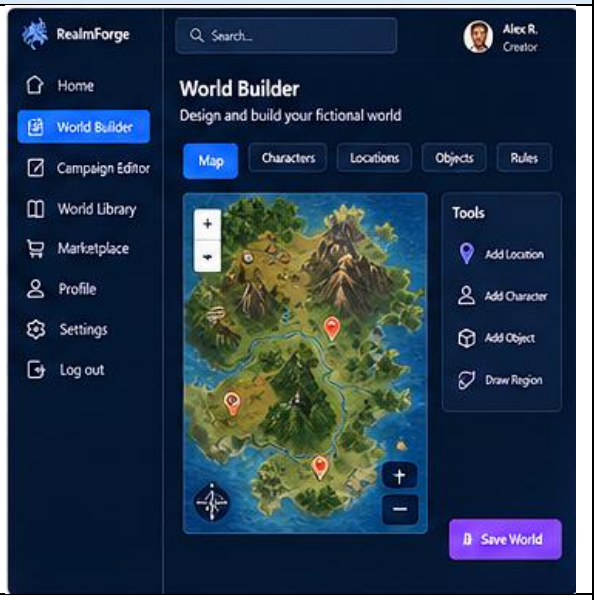
The system records their decisions, generates narrative outcomes, and maintains independent game states. Once the campaign reaches its completion target, the analytics component produces a performance report, and the AI recommendation component suggests improvements.

The filmmaker can then revise the world and publish an updated campaign version.

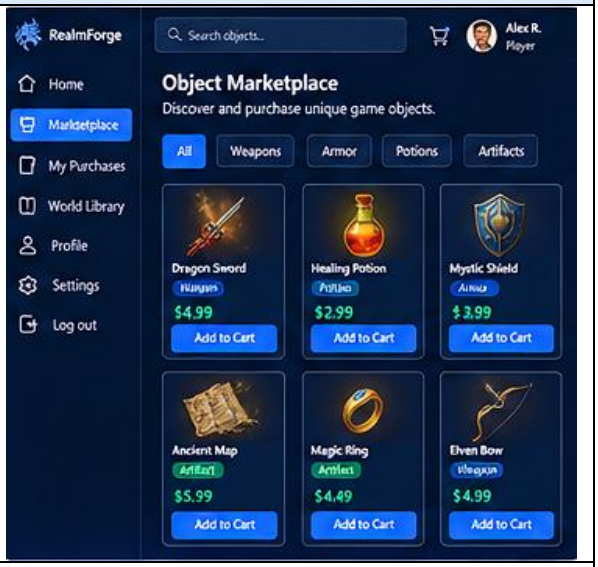
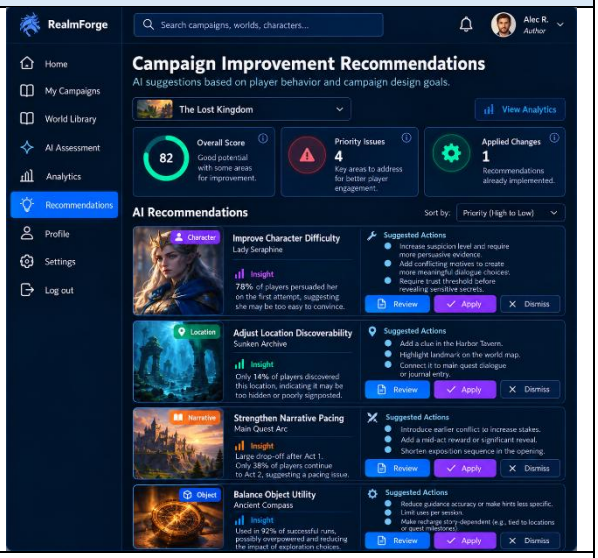
This workflow demonstrates the interaction between content creation, AI-driven gameplay, data storage, behavioral analysis, and continuous campaign improvement.

Prototype Interface Screens

Eight proposed interface concepts are shown below. The mockups share a consistent dark theme and illustrate a simplified design intended to guide a possible Streamlit implementation.

Player Dashboard	AI Game Director / Game Session
	
Player progress, active adventures, and suggested worlds.	Text-based gameplay, player choices, session controls, and game log.
Campaign Author Dashboard	**World Builder / Campaign Editor**
	
Create, manage, and monitor campaigns.	Edit world details, characters, locations, objects, and rules.

Prototype Interface Screens (continued)

AI Campaign Assessment	Object Marketplace
	
Evaluate campaign quality, consistency, and playability	Browse and manage reusable campaign objects.
Campaign Analytics Dashboard	**AI Recommendations**
	
Monitor campaign performance and player engagement.	Provide personalized suggestions for campaigns and gameplay.

Architecture

The two chosen architectures

Layered architecture organizes software into an ordered hierarchy of responsibilities, with each layer providing services through defined interfaces. Our design adopts strict adjacent-layer interaction across Presentation, Application, Domain, and Infrastructure. This structure suits a platform in which authoring, gameplay, commerce, and administration interfaces depend on business rules and technical services that change for different reasons. For example, replacing an AI provider changes authentication, request formats, and response handling, while campaign permissions and gameplay validation should retain their meaning. If these responsibilities were mixed together, a provider change would also require modifying and retesting code responsible for game behavior. The proposed hierarchy separates provider-specific integration from domain decisions and makes their interaction explicit through the intervening interfaces. Similarly, different user interfaces can access shared application capabilities while preserving consistent business rules. These boundaries help contain changes and support focused testing, making Layered a strong candidate for the project's small-team implementation. The benefit depends on maintaining meaningful interfaces and respecting layer boundaries; workflows that cross several responsibilities also require additional coordination through the hierarchy.

Event-Based architecture organizes software as independent components that publish and receive messages asynchronously through an event bus. Its defining characteristics such as indirect communication, subscription-based delivery, and independent processing, fit the platform's need to support several reactions to the same business occurrence. A committed gameplay turn, for example, supplies a player-facing result and information for analytics and milestone reporting. If Gameplay Runtime directly coordinated every downstream use, adding a reporting capability would require changes to gameplay integration code even when gameplay behavior remained unchanged. Publishing a defined event allows interested components to receive that information through subscriptions and process it independently. The same mechanism supports reactions to campaign publication and changes in purchased entitlements. This makes Event-Based a strong alternative where downstream capabilities evolve independently or can complete after the initiating operation. The bus centralizes routing and delivery while business components retain responsibility for decisions and state changes. This reduces direct dependencies, but introduces requirements for correlation, ordering, retries, and duplicate handling. Gameplay operations that depend on AI or persistence results still require coordinated completion; routing those requests through the bus standardizes communication rather than removing that dependency.

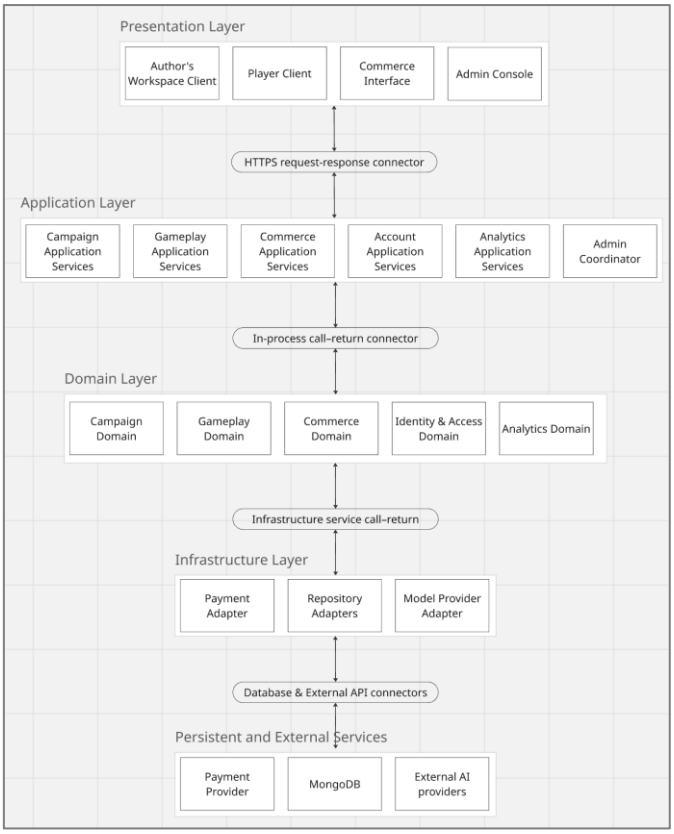


Figure 1- Software design based on layered architecture style

Author's Workspace Client	The interface through which campaign authors upload documents, create and edit fictional worlds, review AI feedback, publish campaigns, and access performance reports and improvement recommendations.
Player Client	The interface through which players browse campaigns, create characters, interact with the AI Game Director through text-based chat, view action outcomes, and resume saved games.
Commerce Interface	The interface through which object creators publish virtual assets and view revenue information, and users browse assets, purchase campaign-specific usage rights, and manage subscriptions.
Admin Console	The interface through which system administrators manage user accounts, content moderation, marketplace activities, and subscription-related issues.
Campaign Application Services	Coordinates campaign creation, document submission, AI evaluation, editing, publication, and lifecycle management. Connects author requests with the domain services responsible for campaign content and publication requirements.
Gameplay Application Services	Coordinates character creation, game startup, saved-game retrieval, and player turns. Organizes the retrieval of campaign information, AI Game Director processing, game-state updates, and delivery of narrative responses.
Commerce Application Services	Coordinates virtual asset creation, marketplace listings, purchases, and the delivery of campaign-specific usage rights. Tracks purchase progress and presents transaction results.
Account Application Services	Coordinates user registration, sign-in, account management, subscription changes, and retrieval of available AI models and features.
Analytics Application Services	Coordinates milestone-based performance reports and AI recommendation jobs. Manages their scheduling, progress, completion, and delivery to campaign authors.

Admin Coordinator	Coordinates administrative requests involving accounts, campaign content, marketplace activities, and subscriptions. Applies authorization checks and directs each request to the relevant domain service.
Campaign Domain	Represents fictional worlds, campaign documents, characters, environments, rules, objects, and relationships. Defines campaign consistency requirements, content revisions, publication eligibility, and completion conditions.
Gameplay Domain	Represents player characters, independent game sessions, inventories, interaction histories, and current game conditions. Implements the AI Game Director's gameplay responsibilities: preparing relevant context, evaluating proposed outcomes against campaign rules, and determining accepted game-state changes.
Commerce Domain	Represents virtual assets, purchases, campaign-specific usage rights, and subscription plans. Defines asset availability, purchase conditions, and access to AI models, analytics, and other subscription features.
Identity & Access Domain	Represents user accounts, account status, roles, and permissions. Defines access rules for players, campaign authors, object creators, and system administrators.
Analytics Domain	Defines structured gameplay events, campaign performance measures, and report contents. Calculates participation, encounters, exploration, and action-outcome statistics, and prepares AI-based campaign improvement recommendations for author review.
Payment Adapter	Provides a common interface to the external payment provider. Translates payment requests into the provider's format and returns transaction references, payment status, and processing errors.
Repository Adapters	Provide storage and retrieval interfaces for workspace documents, campaign definitions, player histories, game states, accounts, marketplace records, and reports. Translate these records into MongoDB operations and perform consistency-preserving updates.
Model Provider Adapter	Provides a common interface to external AI providers such as OpenAI, Anthropic, Google, and Meta. Translates generation requests, manages provider authentication, and returns generated responses or processing errors.
MongoDB	The document-oriented database used to store workspace documents and non-relational application data, including fictional worlds, campaign rules, character information, independent player states, interaction histories, commercial records, and reports.
Payment Provider	The external system that processes payments and supplies transaction references and payment-status information for purchases and subscriptions.
External AI Providers	The external systems that supply model inference for campaign evaluation, interactive gameplay, narrative generation, and campaign improvement recommendations.

Layer Style Component Interaction and Communication Flow

Figure 1 shows how the system's components are organized into four layers and communicate through defined interfaces. The Presentation Layer handles user interactions, the Application Layer coordinates workflows, the Domain Layer manages business rules and decisions, and the Infrastructure Layer provides access to databases and external services. Communication follows the adjacent-layer structure, with requests and results passing through the corresponding interfaces.

For example, when a player submits an action during gameplay, the Player Client sends the request to Gameplay Application Services through the HTTPS request-response connector. The application service coordinates the operation with the Gameplay Domain, which prepares the relevant campaign information, player history, and current game state. The Repository Adapters provide access to stored information, while the Model Provider Adapter communicates with an external AI provider to generate a proposed narrative outcome.

The Gameplay Domain evaluates the proposed outcome against the campaign rules and determines which state changes are acceptable. These changes are saved through the Repository Adapters before the application service returns the confirmed result to the player. This sequence ensures that the generated response is consistent with the saved game state.

Other workflows follow the same architectural structure. For example, Campaign Application Services coordinates campaign creation and publication, while Analytics Application Services manages reporting and recommendation jobs using information maintained by the relevant domain and infrastructure components.

The main advantage of this communication structure is that each layer has clearly defined responsibilities. Changes to external AI providers or database implementations can be handled through infrastructure adapters without directly changing gameplay rules or user interfaces. However, operations that involve several layers require additional coordination and may introduce processing overhead.

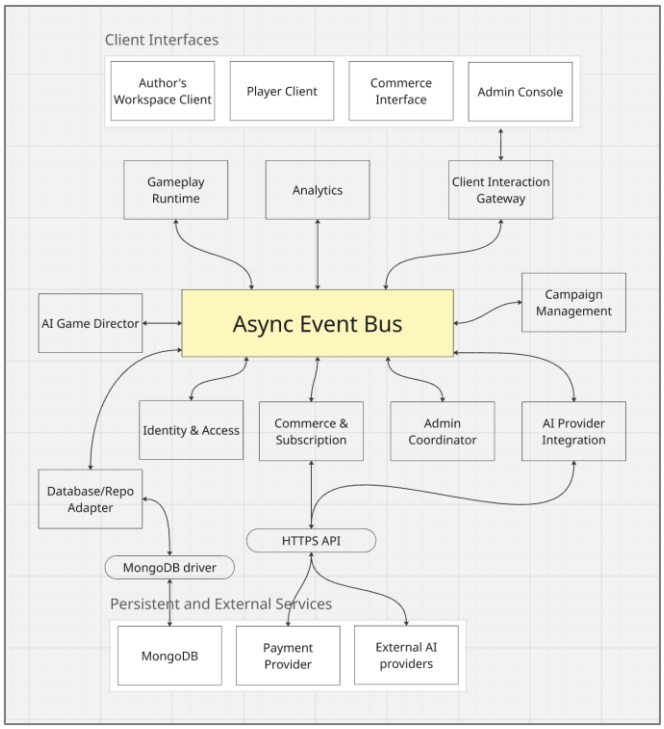


Figure 2- Software design based on event-based architecture style

Event-based Style Component Interaction and Communication Flow

Figure 2 shows an alternative architecture where independent components communicate asynchronously through a central event bus. Instead of directly calling other business components, each component publishes messages and subscribes to the message types it needs to process. The event bus handles message distribution, while individual components remain responsible for their own functionality and state changes.

For example, when a player submits an action, the Client Interaction Gateway receives the request and publishes a GameplayTurnRequested message to the event bus. Gameplay Runtime receives the message, retrieves the relevant session information, and requests AI processing by publishing an AIOutcomeRequested message. The AI Game Director processes this request with assistance from AI Provider Integration, which communicates with the appropriate external language model.

Once AI processing is completed, the proposed outcome is published through the event bus and delivered to Gameplay Runtime. Gameplay Runtime validates the outcome against the campaign rules and coordinates the required game-state updates through the Database/Repo Adapter. After persistence

is confirmed, the system publishes a GameplayTurnCompleted event, allowing the Client Interaction Gateway to return the confirmed result to the player.

Other components can independently subscribe to relevant events. For example, the Analytics component can receive GameplayTurnCompleted events and update campaign performance information without directly interacting with Gameplay Runtime. Campaign Management can similarly publish campaign-related events that other interested components may process.

Each request and its corresponding results must be associated through a correlation identifier, particularly because messages may be processed at different times. The system must also account for delayed responses, duplicate messages, and failed operations to prevent inconsistent game states. Therefore, operations that require validation and persistence confirmation still depend on coordinated completion, even though the communication is asynchronous.

This design reduces direct dependencies between components and makes it easier to introduce new functionality through additional event subscribers. However, it also increases the complexity of monitoring, debugging, and coordinating operations compared with the Layered architecture.

Final Decision

We recommend Layered architecture because it gives our team a clearer way to build and test the platform within the project's scope. Several core operations depend on an ordered sequence of decisions. A gameplay outcome must be validated and saved before the player receives confirmation, and a purchase must be confirmed before usage rights are granted. Application services coordinate these steps through explicit calls, making their execution easier to follow. Event-Based can support the same operations, but requires additional handling for delayed responses, retries, and duplicate messages. The layers also separate business rules from interfaces and external integrations. For example, switching AI providers would mainly affect the provider adapter while leaving gameplay validation intact. This separation helps us make changes and test individual responsibilities without revisiting unrelated parts of the application. Event-Based has a stronger advantage in analytics and milestone reporting, where several components can react independently to gameplay events. However, our immediate priority is to deliver the platform's core functions with manageable development effort. Layered better supports that priority, while Event-Based provides a useful alternative for evaluating the benefits and costs of asynchronous communication. Both designs will be implemented and evaluated as required.

Implementation Plan

Given the limited time and resources available for this group project, we will design the architecture for the full product but implement a minimum viable product (MVP) in both architectural styles. We will defer marketplace transactions, payment processing, subscription management, and advanced administration. The MVP will cover campaign creation and publication, independent gameplay sessions, AI-generated responses, outcome validation, saving and resuming progress, and basic activity reporting. This scope provides a complete author-to-player workflow while allowing us to demonstrate and compare both architectures. We will reuse data models, business rules, and integration code where appropriate, while keeping each architecture's coordination and communication mechanisms distinct.

We will organize reusable code around behavior that remains the same in both architectures, including campaign and session data models, gameplay validation, prompt construction, AI-provider access, database operations, and report calculations. For example, both versions will use the same rules to validate a proposed gameplay outcome but differ in how validation is coordinated with other operations: the Layered version will use calls through defined layer interfaces, while the Event-Based version will exchange messages between components. This approach reduces duplicated work and keeps business behavior consistent, allowing the comparison to focus on the effects of each architecture's communication and coordination mechanisms.

Both architecture implementations will use Python for application logic and Streamlit for user interfaces. They will share data models, business rules, interface elements, and integration code where appropriate. Specific storage libraries and AI-provider tools will be selected during initial development and accessed through adapters to keep those choices separate from the core logic.

Layered Architecture

Stage 1	Establish shared models, validation rules, Streamlit screens, and layer interfaces. Confirm storage and AI-integration choices.
Stage 2	Implement campaign creation and publication, gameplay validation, and saving and resuming sessions.
Stage 3	Add activity reporting, integrate the complete workflow, and test error handling and layer boundaries.
Stage 4	Compare implementations, fix defects, complete diagrams and documentation, and prepare the demonstration.

Event-based Architecture

Stage 1	Define message formats, subscriptions, and request/result correlation. Select the messaging mechanism and verify a basic exchange.
Stage 2	Implement the same campaign and gameplay workflows through event handlers, including AI requests, validation, and persistence confirmation.
Stage 3	Add reporting as an event subscriber. Test delayed results, failed operations, and duplicate-message handling.
Stage 4	Compare implementations, fix defects, complete diagrams and documentation, and prepare the demonstration.